Fitra

Configuration management report

Version 1.1

By: Gary Quah, Matthew Wong, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi

Project Manager (Overall delivery of product)	Gary Quah YiKai
Lead Developer (Overall technical lead, responsible for technical aspects of product release)	Matthew Wong
QA Manager (Overall product and process quality, implementation of QA processes)	Lau Jing Jie
QA Engineer (Devise test plans, conduct tests)	Lee Zheng Wee
Software Configuration Manager (SCM)	Rusyaidi
Release Engineer/Manager (Create baselines and build and integrate changes for delivery)	Quah Yew Yi

This template is the property of Cyrille Michaud. License terms: see http://blog.cm-dm.com/post/2011/11/04/license

Document Change Record

Revision	Description of Change	Approved by	Date
0.10	Initial Template	Gary	15/10/2026
0.20	Add Design Strategies, Architectural Design Patterns, Software Configuration Management Tools	Gary	21/10/2026
0.20	Edit Design Strategies, Architectural Design Patterns, Software Configuration Management Tools	Gary	25/10/2026
1.00	Final Edition	Gary	28/10/2026
1.1	Added Gemini and Strava interface descriptions.	Gary	04/11/2026

TABLE OF CONTENTS

Document Change Record 2
1 Document overview 3
2 Organization 3
2.1 Activities and responsibilities 3
2.1.1 Decisions process and responsibilities 5
3 Configuration identification 5
3.1 Identification rules 5
3.1.1 Identification rules of configuration items 5
3.1.1.1 Identification of a configuration item 5
3.1.2 Identification rules of SOUPs (Software of Unknown Provenance) 6
3.1.3 Identification rules of documents 6
3.1.4 Identification rules of media 7
3.2 Reference configuration identification 7
3.3 Configuration Baseline Management 7
4 Configuration control 9
4.1 Change Management 9
4.2 Interface Management 9
4.3 Evolution's control of SOUP items 11
5 Configuration support activities 12
5.1 Configuration Status Accounting 12
5.1.1 Evolutions traceability 12
5.1.2 Setting up Configuration status 12
5.1.3 Configuration status diffusion 13
5.1.4 Configuration status records storage 13
5.2 Configuration audits 13

5.3 Reviews 13
5.4 Configuration management plan maintenance 14

1 Document overview

This document contains the software configuration management plan for the Fitra software.

2 Organization

The software configuration is managed by members of the project, with specific tools. Responsibilities are shared between

- The software configuration manager (SCM),
- The project manager,
- The technical manager.

2.1 Activities and responsibilities

Describe here the functions required to manage the configuration of the software and responsibilities.

Activities when setting up the project	**Person responsible**
Identify the configuration items	SCM
Install the bug repository tool and set up the database	SCM
Install the software configuration repository tool and set up the database	SCM
Manage and structure the reference space	SCM
Define the configuration processes	SCM

Activities during the project lifecycle	**Person responsible**
Export components for modification, test or delivery	SCM
Set under control validated components	SCM
Create version, write version delivery document	SCM
Approve reference configurations	Project manager
Verify version to be delivered and authorise deliveries	Project manager
Backup spaces	SCM
Do configuration audits	Quality Manager
Inspect configuration records	Quality Manager

Activities during the project lifecycle	Person responsible
Archive reference version	SCM

Management activities	Person responsible
Manage versions and archives	SCM
Manage configuration records	SCM
Produce reports and statistics	SCM
Manage reference space and its access control list	SCM
Manage spaces backup and archive media	SCM
Manage quality reports	Quality Manager

2.1.1 Decisions process and responsibilities

Responsibilities during reviews, audits and approvals are listed below:

At the end of an activity of the project

Activities	Person Responsible
Do a configuration freeze	SCM
Present a configuration state of the components impacted by the activity	SCM
Present a documentation state of the components impacted by the activity	SCM

During a configuration management process audit:

Activities	Person Responsible
Do the configuration management process audit	Project Manager
Present the records of the configuration management process	SCM
Present the quality records of the configuration management process	Quality Manager

Activities	Person Responsible
Present the records of the documentation management process	SCM

3 Configuration identification

3.1 Identification rules

3.1.1 Identification rules of configuration items

3.1.1.1 Identification of a configuration item

The identification of a configuration item is:

FITRA_<COMPONENT_NAME>_V<m.n.p>

- FITRA: project prefix
- <COMPONENT_NAME>: Codebase module identifier (FRONTEND, BACKEND, DB_SCHEMA)
- V<m.n.p>: Version index representing major, minor and patch iterations.

3.1.1.2 Version number of a configuration item

The attribution of a version number is a prerequisite to any delivery of any configuration item. This number shall be incremented before a new delivery if the product or its documentation were modified.

The definition rules of a version number are the following:

- m (Major version): Incremented for major architectural changes, database schema refactoring, or major milestone releases
- n (Minor version): Incremented upon adding new user features (e.g, adding Strava, OAuth, Gemini, Training, etc)
- p (Patch Version): Incremented for bug fixes, performance tuning or hotfixes

3.1.2 Identification rules of SOUPs (Software of Unknown Provenance)

3.1.2.1 Identification of a SOUP

External third-party software, libraries, and managed services are identified using the format: SOUP_<MANUFACTURER/PACKAGE>

3.1.2.2 Version number of a SOUP

SOUP versioning adopts the exact release tag or API version supplied by the package manager in package.json / requirements.txt

3.1.3 Identification rules of documents

3.1.3.1 Description of documents identifiers

The identification of documents is described below:

SC3040_<document type>_<document number>_<revision index>

Where:

" document type " is:

- FOO for documents,
- BAR for bar documents

" document number " is an incremental number, with a separate list for each document type,

" revision index " designates the approved iteration of the document. The revision index is V1 for the first iteration, V2 for the second and so on.

3.1.3.2 Definition and evolution of the revision index

The attribution of a revision index is a prerequisite to any delivery of a document or file. This index shall be incremented before the diffusion of a modified document.

The definition rules of a revision index are the following:

- Drafts and minor edits use decimal versioning, e.g (V0.1, V0.2)
- Final edits of a document use integer increments upon official team approval and sign-offs (V1, V2)

3.1.4 Identification rules of media

Media is, for example, tapes, CD-ROMs, external flash drives or backup cartridges.

3.1.4.1 Internal identification

The identification of media is described below:

<configuration item identification >/<media>/<volume>

where:

"media" is the media number,

"volume" is an incremental number to distinguish the media if the delivery contains more than one media.

3.2 Reference configuration identification

Each reference configuration is defined by:

- An identifier,
- Its content listed in the corresponding Version Delivery Description document,

- The acceptance or validation reviews associated to the building of the reference configuration.

A reference configuration is established for each design review and each test review of the project.

3.3 Configuration Baseline Management

Describe what baselines are to be established. Explain when and how they will be defined and controlled.

Examples of baselines :

- functional baseline (FBL), which describes the system functional characteristics;
- allocated baseline (ABL), which describes the design of the functional and interface characteristics,
- product baseline (PBL), which consists of completed and accepted system components and documentation that identifies these products.

Baseline name	Target milestone	Included configuration items & description	Description
Functional baseline	Post-lab 1 review	Use case model	Defines high-level project goals and scope
		Project proposal	Defines high-level project goals and scope
		Team information	Roles and operational context
Allocated baseline	Post-lab 2 review	System Requirement Specification (SRS)	Detailed functional and interface specifications
		Quality plan	Quality assurance criteria allocated across the project
Allocated baseline, product baseline	Post-lab 3 review	Project plan	Detailed schedule and milestones
		Risk management plan	Risk identification and mitigation strategies (ABL)
		Demo	Functional build of system components (PBL)
Allocated baseline	Post-lab 4 review	Design report on software maintainability	Architectural maintainability design choices

		Configuration management plan	Rules for checking and baseline control
		Change management plan	Procedures for managing changes to baselines
		Release plan	Strategy and schedule for product deployments
Product baseline	Post-lab 5 review	Presentation slides	Summary of work across past 5 labs
		Test plan & test cases	Comprehensive verification procedures
		Coverage report	Test execution metrics validating code completeness
		Requirements test	Verification traces against requirements

4 Configuration control

Configuration control maintains system integrity by introducing a systematic process for managing and regulating changes to all Software Configuration Items (SCI) throughout the software development life cycle (SDLC). SCIs must go through formal technical review before they can be committed to the baseline; changes are strictly not allowed unless authorised by the Change Control Board (CCB). This is to ensure that every change is intentional and tested.

4.1 Change Management

Describe the process for controlling changes to the baselines and for tracking the implementation of those changes.

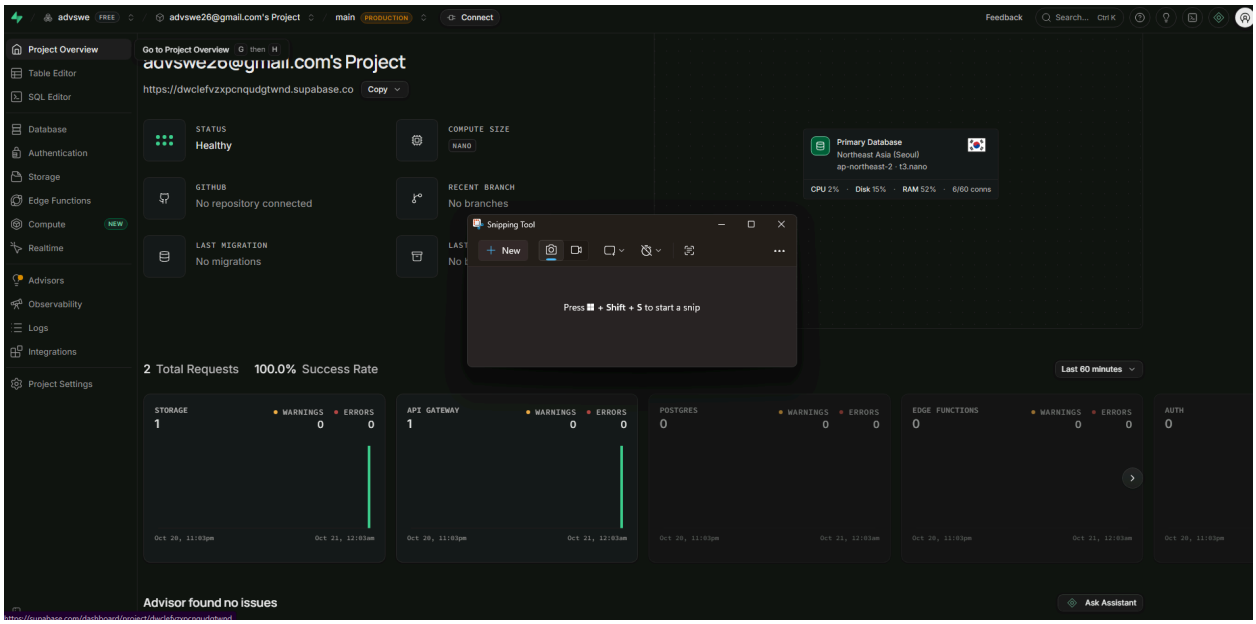
Example for problem resolution:

- Change requests are issued by the Project Manager or Lead Developer according to the problem resolution process.
- When a change request is accepted by the project manager/product manager following CCB review, a branch is created in the SCM
- Release Manager & SCM will verify the branch against the product building process to ensure an accurate, repeatable, and documented build before authorising to baseline integration

4.2 Interface Management

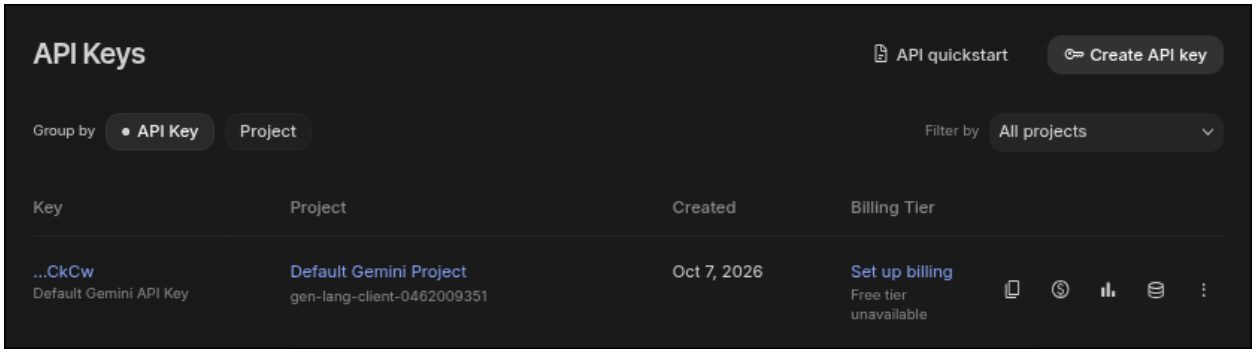
Identify the interfaces to be managed and describe the procedures for identification of interface requirements.

Supabase



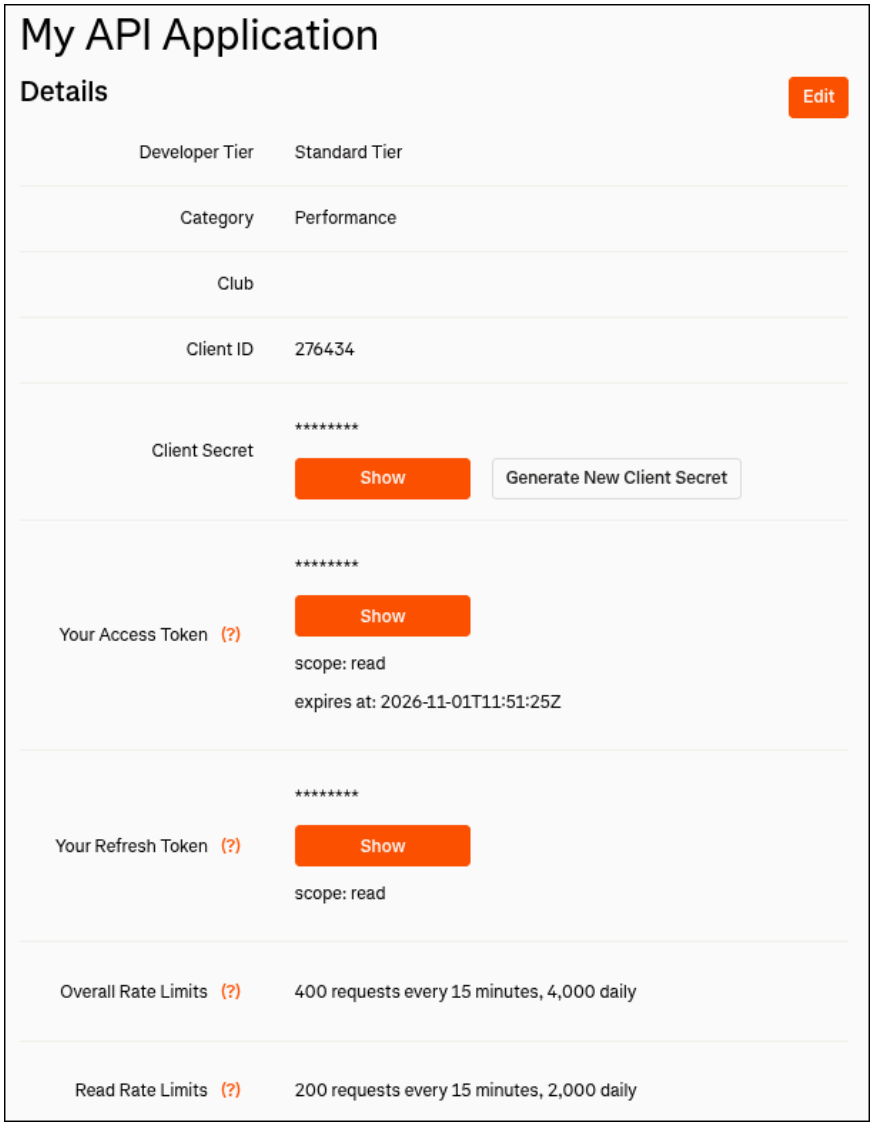
The primary navigation interface for Supabase is located on the left-hand side. The main tab to be used is the “Table Editor” tab where users can view, access and modify table records.

Gemini



The primary navigation interface for G emini would be creating and accessing the API key unde Google AI Studio

Strava



The primary interface for Strava would be the API application page accessed through the strava webpage or strava application. This page allows users to configure their secret and refresh token.

4.3 Evolution's control of SOUP items

Evaluation of Upgrades	Routine dependency updates will not be performed ad hoc. Upgrading a SOUP component requires creating a dedicated maintenance issue/CR (chore/deps-<package name>-upgrade) The upgrade must be tested against the full integration suite to verify that it does not compromise system stability.

Vulnerability Monitoring	Automated security auditing tools monitor frozen SOUP items for common vulnerabilities and exposures (CVEs) If a critical vulnerability is identified, a hotfix CR is generated to patch or replace the compromised library with a minimum viable secure version

5 Configuration support activities

5.1 Configuration Status Accounting

Configuration Status Accounting (CSA) is the process to record, store, maintain and report the status of configuration items during the software lifecycle. All software and related documentation should be tracked throughout the software life. For this project, the records are held in the GitHub Repository and in the team JIRA project, so that the state of any configuration item, the changes applied to it, and the reason for each change can be recovered at any point in the project.

5.1.1 Evolutions traceability

The traceability of modifications of items given their types:

- Document: the Document Change Record on the first page of each document identifies the origin of each modification, its approver and its date. Google Docs retains the full editing history of the document, so the author of any individual change can be identified, and modified paragraphs can be compared against any earlier revision.
- Source file: each commit message records the change made and references the Jira issue that authorised it. The repository therefore holds, for each file, the complete sequence of changes, their authors and their dates.
- Configuration item: the Version Delivery Description of each released version identifies the Jira issues included in that version and the commit range it was built from.

The Jira issue serves as the modification sheet for this project. It describes the modification to be made with enough precision to identify the affected components, records who requested it and who approved it, and is linked to the pull request that implements it.

5.1.2 Setting up Configuration status

The SCM sets up the state of all versions and of each configuration item with:

- The label, which is the annotated Git tag of the version,
- The version number, following the rules in section 3.1.1,
- The creation date of the VDD,
- The list of Jira issues closed in that version,
- The versions of every SOUP resolved by the build, taken from the committed lock files.

5.1.3 Configuration status diffusion

The SCM writes the VDD and the quality manager reviews it. The VDD is committed to the repository alongside the release and is circulated to the team at the sprint review, so that every member knows what is contained in the current version.

5.1.4 Configuration status records storage

The records are stored in the GitHub repository, which is the configuration management system for this project. The repository contains:

- The change requests, as Jira issues referenced by identifier from the commit history,
- The software documents, in the project documentation folder,
- The Version Delivery Descriptions, one per released version,
- The configuration states, as the annotated tags and releases, which are ordered chronologically by the repository itself,
- The dependency lock files that fix the SOUP versions of each build.

5.2 Configuration audits

Configuration audits are carried out by the quality manager and are reported to the project manager at the sprint review, as set out in the software quality assurance plan. Two kinds of audit are performed.

Peer review audit: performed continuously as part of the pull request process. Every change is reviewed by a member who did not write it, who checks that the branch is correctly identified, that the change references a Jira issue, that unrelated changes have not been included, and that any change to a shared configuration item has been announced to the team.

Baseline audit: performed before each baseline is established. The quality manager verifies that the items listed in the Version Delivery Description are the items actually present at the tagged commit, that every document in the baseline carries a current revision index and an approver, and that no change has reached the main branch without a pull request.

Findings from either audit are raised as Jira issues and are tracked to closure in the same way as defects.

5.3 Reviews

The technical reviews of the project are held at the laboratory sessions, and each one closes a project stage. A review is the point at which a baseline is established: the requirements review establishes the functional baseline, the design review establishes the allocated baseline, and the prototype demonstration and final delivery establish and then update the product baseline, as described in section 3.3.

Before each review, the SCM freezes the configuration, meaning that no further change is merged to the main branch until the review has been held, and prepares the configuration state and the Version Delivery Description of the items to be reviewed. During the review, the SCM presents that

state, and the quality manager presents the audit findings. After the review has been passed, the SCM applies the tag that records the baseline and development resumes on new branches taken from it.

5.4 Configuration management plan maintenance

This plan is maintained by the SCM and is approved by the project manager. It is reviewed at the start of each sprint and is updated whenever a change is made to the identification rules, the roles and responsibilities, the change management process, or the set of tools used to manage the configuration.

Any team member may propose a change to this plan by raising a Jira issue. An accepted change is applied by the SCM, the revision index of this document is incremented, and the change is recorded in the Document Change Record on the first page, together with its approver and date. The current revision of this plan is held in the project documentation folder of the repository, and the revision in force is the one referenced by the most recent reference configuration.